

Docker Week 4 Practical Report

Student: Rajesh Kumar

Environment: Ubuntu Linux on WSL2 with Docker Engine

Repository: week-4-docker-practical

1. What is Docker?

Docker is a containerization platform used to package applications and their dependencies into portable, isolated runtime environments called containers. Containers provide process and filesystem isolation while sharing the host operating system kernel, making application deployment more consistent.

2. Docker Architecture

Docker uses a client-server architecture. The Docker CLI sends commands to the Docker Engine. The Engine manages images, containers, networks, and volumes. Docker registries such as Docker Hub provide storage and distribution of images.

3. Images vs Containers

A Docker image is a read-only template used to create containers. A container is a running instance of an image. Images can be tagged and distributed, while containers can be started, stopped, restarted, inspected, and removed.

4. Dockerfile

A Dockerfile contains instructions for building a Docker image. This project uses Nginx Alpine as the base image, copies index.html into the Nginx web root, and documents port 80.

5. Docker Volumes

Docker volumes provide persistent storage independent of a container's lifecycle. The project created a named volume called week4-data, stored a test file in it, removed the original container, and successfully read the same data using a new container.

6. Docker Networks

Docker networks provide controlled communication between containers. A custom bridge network called week4-network was created and the web container was connected to it. Network inspection confirmed the connection.

7. Docker Compose

Docker Compose defines and runs multi-container applications using YAML. This project uses a web service built from the Dockerfile and a Redis service based on redis:alpine. Both services use a shared custom network, while the web service uses a named log volume.

8. Docker Hub

Docker Hub is a registry for storing and distributing container images. The custom image was tagged as rajeshkumar357/week4-docker-app:v1 and successfully pushed to a public Docker Hub repository.

9. Practical Implementation

Exercise	Implementation
Docker verification	docker --version, docker info, docker run hello-world
Image	week4-docker-app:v1 built from Dockerfile
HTML application	Nginx serving index.html on localhost:8080
Volume	week4-data persistence test
Network	week4-network custom bridge network
Compose	Nginx + Redis multi-container deployment
Docker Hub	rajeshkumar357/week4-docker-app:v1 pushed successfully

10. Evidence Checklist

No.	Evidence	Status
01	Docker installation	Add actual screenshot
02	Docker verification / hello-world	Add actual screenshot
03	Docker images / build	Add actual screenshot
04	Running HTML application and container	Captured
05	Custom Docker network	Captured
06	Persistent Docker volume	Captured
07	Docker Compose with Nginx + Redis	Captured
08	Docker Hub image/repository	Captured

Evidence integrity: This report does not claim screenshots that were not provided. Screenshots 04–08 were captured during the practical sessions. Screenshots 01–03 must be added from the user's actual Docker installation, verification, and image-build/pull sessions before final submission.

11. Conclusion

The practical demonstrates an end-to-end Docker workflow: verifying Docker Engine, obtaining images, managing containers, building a custom image, serving an HTML application, managing persistent storage and networking, orchestrating multiple services with Docker Compose, and publishing an image to Docker Hub.